

# Subqueries, CTEs and views

How do I break a hard question into named steps, and save them for Power BI?

BY THE END OF TODAY, YOU CAN

# Class objectives

---

**01** Answer a two-step question with a subquery

---

**02** Rewrite a nested query as a CTE

---

**03** Save a query as a view

---

**04** Explain what a view stores and costs

---

**05** Prove a view with a known total

# Warm-up

Five questions on D07, ungraded. The fifth is the reason for today.

---

Suggested time • 10 minutes

# Subqueries: one query inside another

A number or a list the outer query needs, worked out fresh every time.

---

# A question in two steps

“Which completed orders are bigger than our average completed order?”

---

Suggested time • 12 minutes

[labs/D08/01-Ins-Two-Steps/README.md](#)

# First, find the average

One row is one \_\_\_\_? Predict, then run.

```
SELECT ROUND(AVG(order_total), 2) AS average_order  
FROM nakheel.orders  
WHERE status = 'completed';
```

→ **1 row: 114.05**

# Then compare every order with it

**114.05** is typed in by hand, copied from step 1.

```
SELECT order_id, customer_id, order_total
FROM nakheel.orders
WHERE status = 'completed'
      AND order_total > 114.05
ORDER BY order_total DESC;
```

→ **689 rows · order 100849 first, 1,964.50**

WHAT IS WRONG WITH THIS?

# A typed number goes stale

A new order arrives tomorrow and the average changes. The typed query does not.

**Step 1, run today**

**114.05**

worked out by the query

**Step 2, typed by hand**

**114.05**

unchanged tomorrow

## DEFINITION

**A subquery** is a query inside another query, in brackets. The inner one runs first; the outer one uses its answer.

# Put step 1 where its answer is needed

The inner query takes the place of 114.05.

```
SELECT order_id, customer_id, order_total
FROM nakheel.orders
WHERE status = 'completed'
      AND order_total > (SELECT AVG(order_total)
                        FROM nakheel.orders
                        WHERE status = 'completed')
ORDER BY order_total DESC;
```

→ 689 rows again · order 100849 first

**Run the inner query on its own first.** If it does not give the number you expect, the outer query cannot be right.

# A list from another table, with IN

“How many completed orders came from wholesale customers, and what were they worth?”

---

Suggested time • 13 minutes

[labs/D08/02-Evr-Subquery-In/README.md](#)

# Which customers are wholesale?

Predict the row count, then run.

```
SELECT customer_id  
FROM nakheel.customers  
WHERE LOWER(TRIM(segment)) = 'wholesale';
```

→ 65 rows: a list, not one number

# Use the list with IN

**IN** keeps a row when its value is in the list. **=** needs one value.

```
SELECT COUNT(*) AS completed_orders,  
       ROUND(SUM(order_total), 2) AS revenue  
FROM nakheel.orders  
WHERE status = 'completed'  
      AND customer_id IN (SELECT customer_id  
                          FROM nakheel.customers  
                          WHERE LOWER(TRIM(segment)) = 'wholesale');
```

→ **377 orders · 154,204.00**

# Change IN to =, and run

= asks for one value. The inner query returns 65.

```
SELECT COUNT(*) AS completed_orders,  
       ROUND(SUM(order_total), 2) AS revenue  
FROM nakheel.orders  
WHERE status = 'completed'  
      AND customer_id = (SELECT customer_id  
                        FROM nakheel.customers  
                        WHERE LOWER(TRIM(segment)) = 'wholesale');
```

Scalar subquery produced more than one element

**Put IN back. Use IN for a list.**

# Subquery or join? The same answer

## Subquery with IN

→ 377 · 154,204.00

```
SELECT COUNT(*) AS completed_orders,  
       ROUND(SUM(order_total), 2) AS revenue  
FROM nakheel.orders  
WHERE status = 'completed'  
      AND customer_id IN (SELECT customer_id  
                          FROM nakheel.customers  
                          WHERE LOWER(TRIM(segment)) = 'wholesale');
```

## The D07 join

→ 377 · 154,204.00

```
SELECT COUNT(*) AS completed_orders,  
       ROUND(SUM(o.order_total), 2) AS revenue  
FROM nakheel.orders AS o  
INNER JOIN nakheel.customers AS c  
  ON o.customer_id = c.customer_id  
WHERE o.status = 'completed'  
      AND LOWER(TRIM(c.segment)) = 'wholesale';
```

Only filtering? IN often reads more easily.

STUDENT DO

# Activity: Subquery practice

In this activity, you will answer four manager questions with a subquery, including what never matched.

Suggested time

**12 minutes**

[labs/D08/03-Stu-Subquery-Practice/README.md](#)

# Activity: Subquery practice

## INSTRUCTIONS

1. Which products cost more than the average list price?
2. Which customers have never placed an order? Use NOT IN.
3. Which products have never been sold?
4. Who is our biggest customer by completed revenue?

## HINT

Run the inner query on its own first, every time.

## BONUS

NOT IN when the list holds a NULL: the Bonus in the README.

# Time's up. Let's review.

Subquery practice

---

Solved: `labs/D08/03-Stu-Subquery-Practice/Solved/subquery_practice.sql`

**NOT IN** returns no rows at all when the inner list contains a NULL. If it could, use D07's LEFT JOIN ... IS NULL.

# CTEs: name each step

The same steps, written top to bottom with names.

---

# From nested to named

“What does a typical customer spend, in each segment?”

---

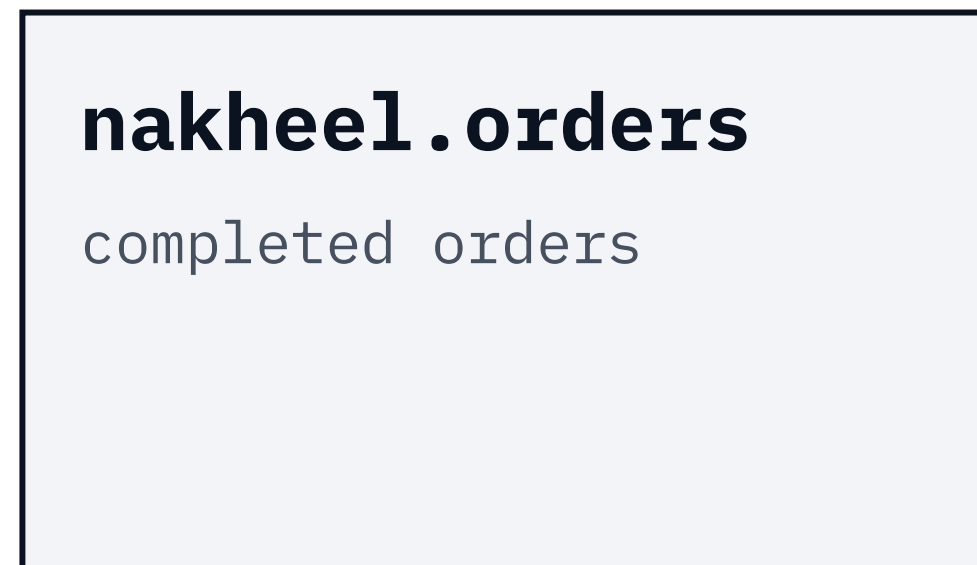
Suggested time • 12 minutes

[labs/D08/04-Ins-Nested-To-Named/README.md](#)

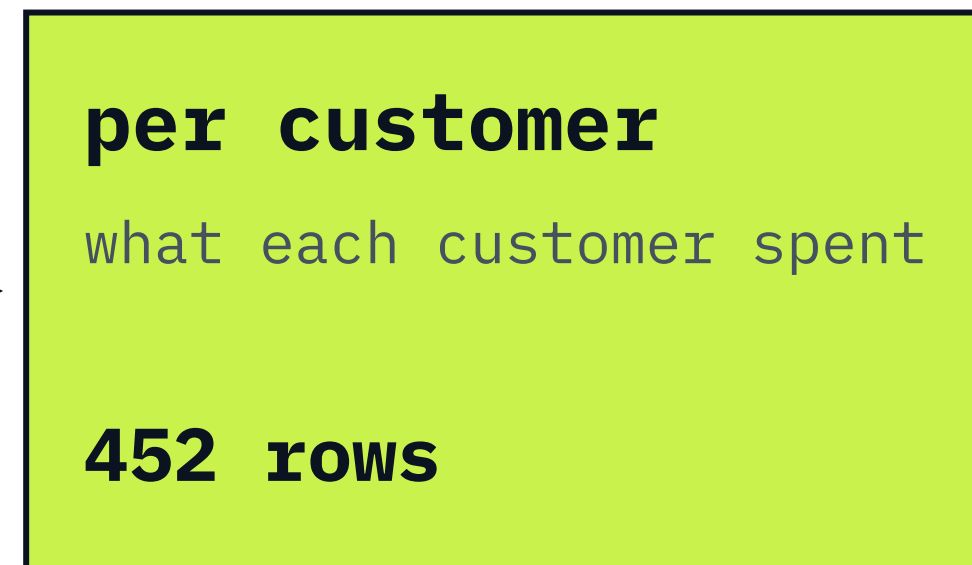
# One GROUP BY cannot answer it

First one row per customer: what each spent. Then one row per segment: the average of those.

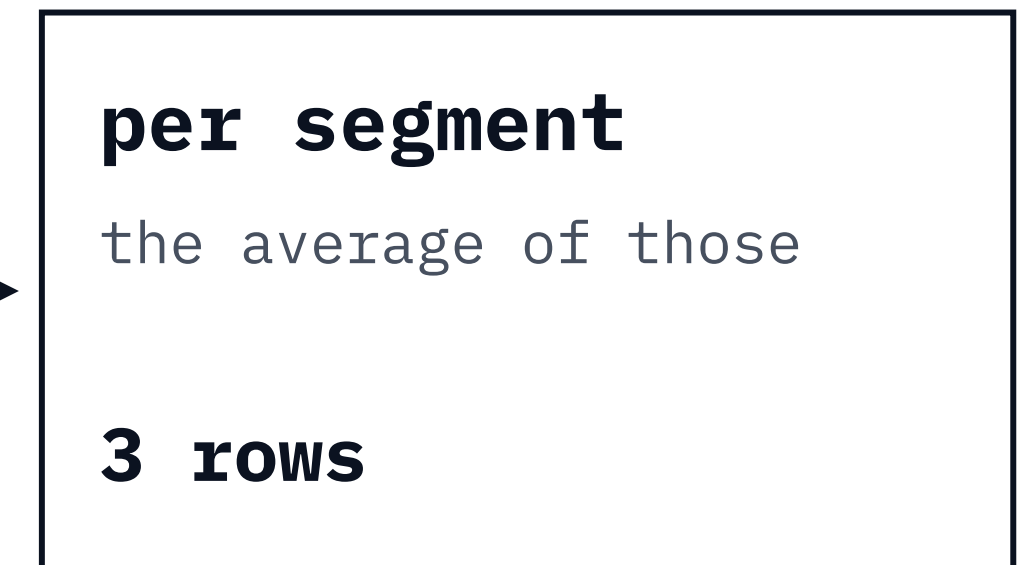
START



STEP 1



STEP 2



# What each customer spent

One row is one \_\_\_\_? Predict the row count.

```
SELECT customer_id,  
       COUNT(*) AS completed_orders,  
       SUM(order_total) AS revenue  
FROM nakheel.orders  
WHERE status = 'completed'  
GROUP BY customer_id;
```

→ **452 rows**

# Step 1 in brackets, inside step 2

It starts with the answer's columns, then dives into brackets for the part that runs first.

```
SELECT COALESCE(LOWER(TRIM(c.segment)), 'unknown') AS segment_clean,  
       COUNT(*) AS customers,  
       ROUND(AVG(cr.revenue), 2) AS avg_revenue_per_customer  
FROM (SELECT customer_id, COUNT(*) AS completed_orders, SUM(order_total) AS revenue  
      FROM nakheel.orders  
      WHERE status = 'completed'  
      GROUP BY customer_id) AS cr  
LEFT JOIN nakheel.customers AS c  
      ON cr.customer_id = c.customer_id  
GROUP BY segment_clean  
ORDER BY avg_revenue_per_customer DESC;
```

→ 3 rows

## DEFINITION

**A CTE (common table expression)** is a named step at the top of a query. The SELECT after it reads the name like a table.

# WITH gives step 1 a name

**WITH customer\_revenue AS ( ... )** runs step 1 and names its result.

```
WITH customer_revenue AS (  
    SELECT customer_id,  
           COUNT(*) AS completed_orders,  
           SUM(order_total) AS revenue  
    FROM nakheel.orders  
    WHERE status = 'completed'  
    GROUP BY customer_id  
)  
SELECT COALESCE(LOWER(TRIM(c.segment)), 'unknown') AS segment_clean,  
       COUNT(*) AS customers,  
       ROUND(AVG(cr.revenue), 2) AS avg_revenue_per_customer  
FROM customer_revenue AS cr  
LEFT JOIN nakheel.customers AS c  
    ON cr.customer_id = c.customer_id  
GROUP BY segment_clean  
ORDER BY avg_revenue_per_customer DESC;
```

# The **SELECT** reads the name like a table

Top to bottom, in the order it runs.

```
WITH customer_revenue AS (  
    SELECT customer_id,  
           COUNT(*) AS completed_orders,  
           SUM(order_total) AS revenue  
    FROM nakheel.orders  
    WHERE status = 'completed'  
    GROUP BY customer_id  
)  
SELECT COALESCE(LOWER(TRIM(c.segment)), 'unknown') AS segment_clean,  
       COUNT(*) AS customers,  
       ROUND(AVG(cr.revenue), 2) AS avg_revenue_per_customer  
FROM customer_revenue AS cr  
LEFT JOIN nakheel.customers AS c  
    ON cr.customer_id = c.customer_id  
GROUP BY segment_clean  
ORDER BY avg_revenue_per_customer DESC;
```

THE ANSWER · 3 ROWS

# Wholesale: about six and a half times retail

The D07 totals hid this: wholesale had fewer customers but nearly half the revenue.

| segment_clean | customers | avg_revenue_per_customer |
|---------------|-----------|--------------------------|
| wholesale     | 58        | 2,658.69                 |
| unknown       | 1         | 938.00                   |
| retail        | 393       | 409.37                   |

# A two-step pipeline

“Which months were better than our average month?” Plan the steps aloud before typing.

---

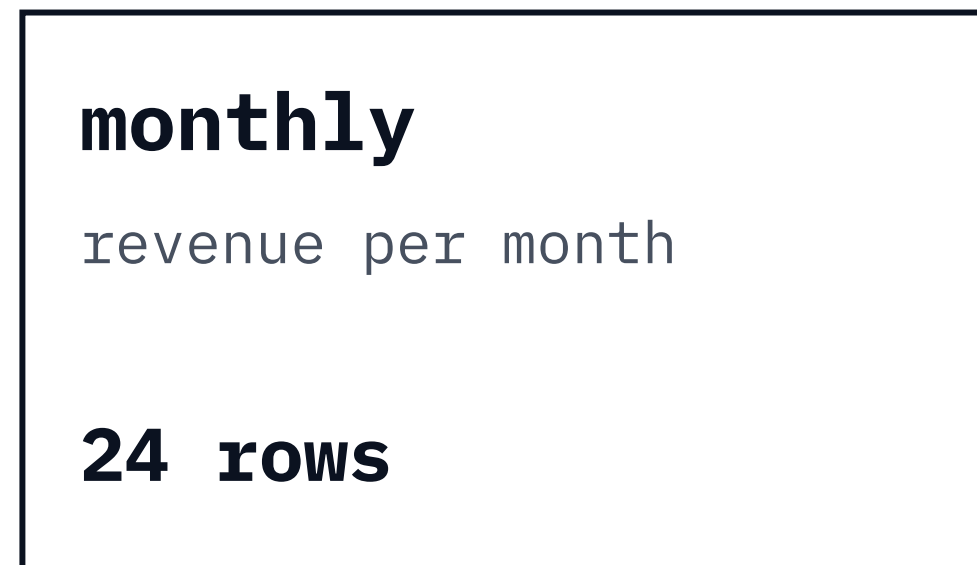
Suggested time • 15 minutes

[labs/D08/05-Evr-Two-CTE-Pipeline/README.md](https://github.com/100xdevs/labs/blob/main/D08/05-Evr-Two-CTE-Pipeline/README.md)

# Three steps, each with a name

Each step reads the one before it. Check its rows before adding the next.

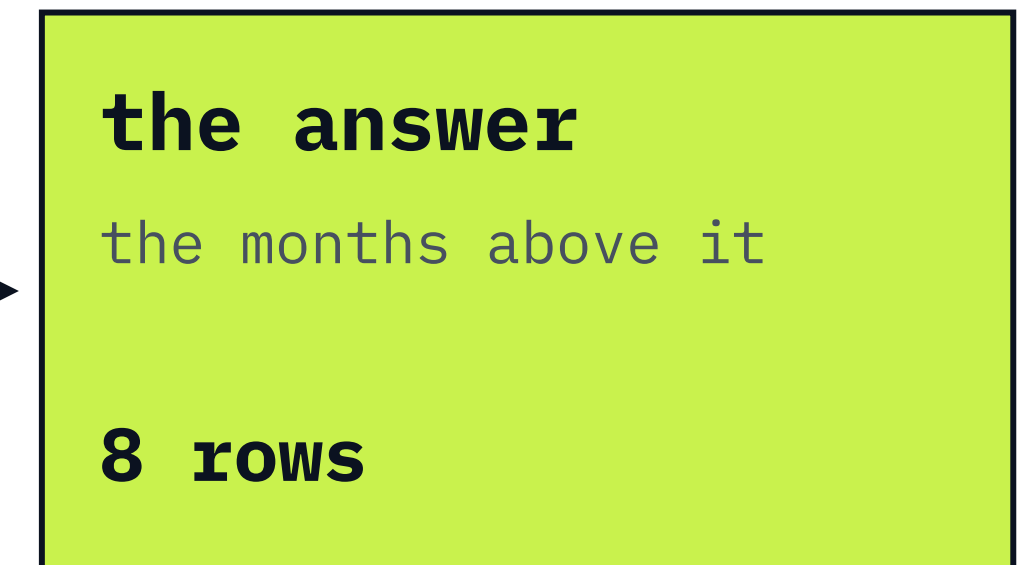
STEP 1



STEP 2



STEP 3



# Revenue per month, checked on its own

SELECT \* FROM monthly shows step 1 before anything reads it.

```
WITH monthly AS (  
  SELECT DATE_TRUNC(order_date, MONTH) AS month,  
         SUM(order_total) AS revenue  
  FROM nakheel.orders  
  WHERE status = 'completed'  
  GROUP BY month  
)  
SELECT *  
FROM monthly  
ORDER BY month;
```

→ 24 rows · September 2024 first, 11,268.00

## A second step reads the first

After a comma, the second CTE. Check it alone.

```
average_month AS (  
    SELECT AVG(revenue) AS avg_revenue  
    FROM monthly  
)
```

→ 1 row: 13,167.65

# One number, used as a subquery

Step 2 is one number, so > can use it: the 2.1 idea again.

```
SELECT month,  
       ROUND(revenue, 2) AS revenue  
FROM monthly  
WHERE revenue > (SELECT avg_revenue FROM average_month)  
ORDER BY month;
```

→ 8 rows • April 2026 the best, 20,237.00

STUDENT DO

# Activity: One row per customer

In this activity, you will build a customer summary in two named steps, for every customer, and audit it.

Suggested time

**18 minutes**

[labs/D08/06-Stu-Customer-Summary/README.md](#)

# Activity: One row per customer

## INSTRUCTIONS

1. Per customer: completed orders, revenue, first and last order date.
2. Every customer, including those with no orders.
3. Audit: customers, orders, revenue, and how many have none.

## HINT

Step 1 is one row per customer who ordered. Step 2 keeps every customer.

## NEW

COUNTIF counts the rows where a test is true. It is a BigQuery function.

# Time's up. Let's review.

One row per customer

---

Solved: `labs/D08/06-Stu-Customer-Summary/Solved/customer_summary.sql`

# Why 315,085.50 and not 316,023.50?

The summary: 500 customers, 2,770 orders, 49 with none.

**From customers**

**315,085.50**

**On orders**

**316,023.50**

938.00 is order 101664: its customer is not in customers, so it has no row here.

# Views: save the steps

A query saved in the dataset with a name, for Power BI to read.

---

# Save a query as a view

Create the bi dataset, save one query in it, and read it like a table.

---

Suggested time • 15 minutes

[labs/D08/07-Ins-First-View/README.md](https://github.com/DataCamp/labs/blob/main/D08/07-Ins-First-View/README.md)

WHY SAVE IT

# A CTE lives for one query

These should use the result by name, without copying the SQL.

## Power BI

reads it from D17

## A colleague

without your SQL

## Tomorrow's you

the same answer, fresh

## DEFINITION

**A view** is a query saved in the dataset with a name. It stores no rows: every read runs the query again.

# Create the bi dataset

01 ⋮ next to your project → Create dataset

02 Dataset ID: bi

03 Location: the same as nakheel (Multi-region US)

04 Create dataset

**A view must be in the same location as the tables it reads.**

# Save the D06 join as a view

Everything after AS is an ordinary query: the D06 join.

```
CREATE OR REPLACE VIEW bi.fact_sales AS
```

```
SELECT i.item_id,  
       i.order_id,  
       o.order_date,  
       o.channel,  
       o.status,  
       o.customer_id,  
       i.product_id,  
       i.qty,  
       i.price,  
       i.qty * i.price AS revenue  
FROM nakheel.order_items AS i  
INNER JOIN nakheel.orders AS o  
  ON i.order_id = o.order_id;
```

→ **view created**

one row per  
order line

every status  
kept

# What you see in the explorer

- 
- 01 A view icon: fact\_sales under bi
  - 02 Schema: its 10 columns
  - 03 Details: the saved SQL
  - 04 No Preview rows and no storage size
-

READ IT LIKE A TABLE

# The audit, from the view

The same numbers as D06.

```
SELECT COUNT(*) AS line_rows,  
       COUNT(DISTINCT order_id) AS different_orders,  
       ROUND(SUM(revenue), 2) AS all_revenue,  
       ROUND(SUM(CASE WHEN status = 'completed' THEN revenue END), 2) AS completed_revenue  
FROM bi.fact_sales;
```

→ 7,516 · 3,000 · 340,370.00 · 316,023.50

# 0 rows



stored. Every read runs the join again and is billed. On billions of rows, a view everyone reads is a view everyone pays for.

Plan 4.1 · in the sandbox, reads count against the 1 TiB monthly allowance

# The customer and product views

One table of events, and small tables that describe it: the D07 cleaning rules, saved once.

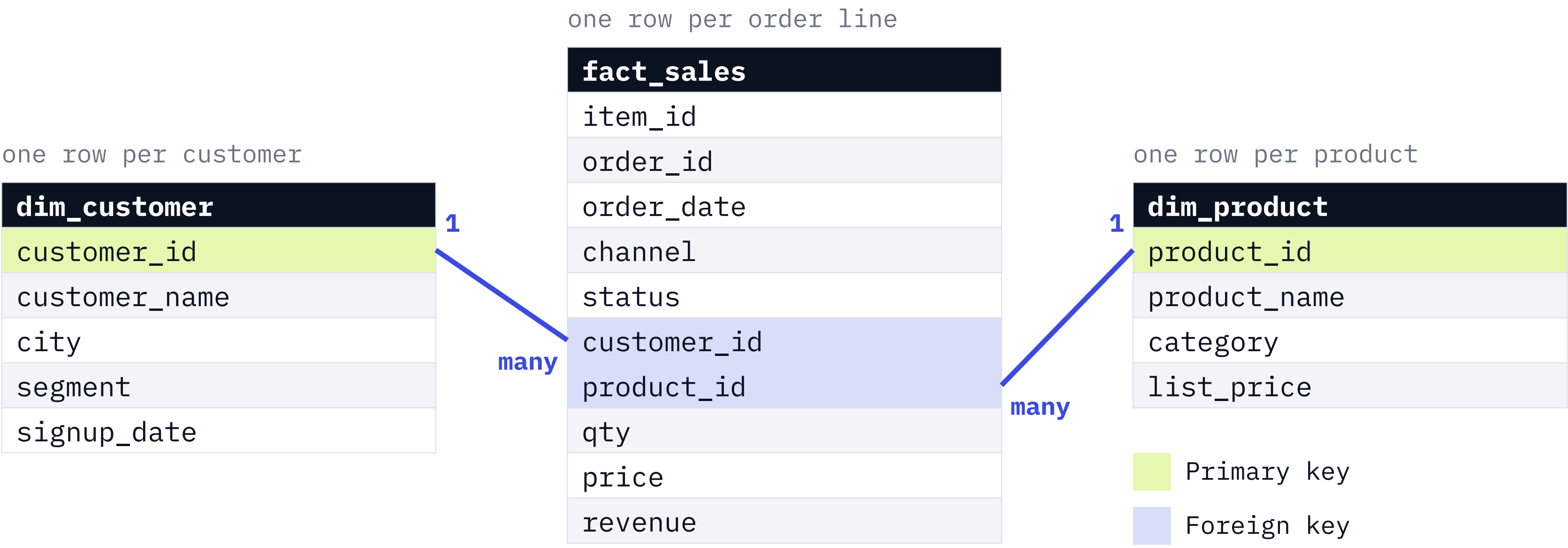
---

Suggested time • 20 minutes

[labs/D08/08-Evr-Dimension-Views/README.md](#)

# fact\_sales in the middle, what describes it around

Every report joins fact\_sales to these views on the ID.



## D07's city rule, saved once

The CASE is D07 3.2's city rule; the segment line is 3.1's cleaning.

```
CREATE OR REPLACE VIEW bi.dim_customer AS
SELECT customer_id,
       customer_name,
       CASE
         WHEN city IS NULL THEN 'unknown'
         WHEN LOWER(TRIM(city)) = 'al zarqa' THEN 'zarqa'
         ELSE LOWER(TRIM(city))
       END AS city,
       LOWER(TRIM(segment)) AS segment,
       signup_date
FROM nakheel.customers;
```

CHECK EACH VIEW

# Rows equal different IDs

Check 3 of the Five Checks: the ID must be unique here, or every join to fact\_sales fans out.

| view            | rows | different IDs | groups                |
|-----------------|------|---------------|-----------------------|
| bi.dim_customer | 500  | 500           | 2 segments · 9 cities |
| bi.dim_product  | 60   | 60            | 6 categories          |

# Time's up. Let's review.

Subquery, CTE or view?

---

No Solved file: the comparison, then three quick questions in the chat

# Give a step a place to live

|          | Where it lives                       | Lasts                                      | Use it for                                              |
|----------|--------------------------------------|--------------------------------------------|---------------------------------------------------------|
| Subquery | inside one query, in brackets        | while the query runs                       | one number or one list the outer query needs            |
| CTE      | at the top of one query, with a name | while the query runs                       | a question in several steps, each with its own grain    |
| View     | in the dataset, with a name          | until you drop it (60 days in the sandbox) | a query other queries, people and Power BI read by name |

# Subquery, CTE or view?

- 
- 01 The average order value inside a WHERE?
  - 02 Customer totals, then the average per segment, in one report?
  - 03 The cleaned customer list Power BI reads every morning?

# Break

 Back at 7:00 PM

---

After the break: prove the views give the numbers we know, then the lab. Ask in your group channel.

# Lab: views we can trust

Prove the views with numbers we already know, then add three more.

---

# Prove the views give the numbers we know

Three checks against D06 and D07. The joins are shorter now: fact\_sales carries the status.

---

Suggested time • 20 minutes

[labs/D08/09-Evr-Check-The-Views/README.md](#)

# The views give the numbers we know

| Check                     | Result                                                 | Known from                |
|---------------------------|--------------------------------------------------------|---------------------------|
| 1 · by category           | Outerwear first, 76,637.00                             | the D06 lab               |
| 2 · by segment, LEFT JOIN | retail 160,881.50 · wholesale 154,204.00 · NULL 938.00 | D07 3.1                   |
| 3 · INNER JOIN audit      | 6,946 lines · 315,085.50                               | the orphan's 5 lines gone |

Power BI meets order 101664 too: on D18, those 5 lines have no customer. The lab's C3 fixes it at the source, in a view.

# Launch the lab

Three more views, each with an audit.

---

Suggested time • 5 minutes

[labs/D08/10-Lab-Reporting-Views/README.md](#)

# Reporting views for Power BI and for D10

## TASK

Three more views in bi, each with an audit.

## FILES

labs/D08/10-Lab-Reporting-Views/README.md

Unsolved/reporting\_views.sql

## DONE WHEN

Each view's audit matches the numbers in the README.

---

In class until 7:50; finish after class.

# Lab: Reporting views for Power BI and for D10

In this lab, you will save three more views with an audit each: the customer summary D10 starts from, category by month, and a complete customer list.

Suggested time

**25 minutes**

[labs/D08/10-Lab-Reporting-Views/README.md](#)

# Lab: Reporting views for Power BI and for D10

## CHECKPOINTS

C1 bi.customer\_summary: 500 customers, 2,770 orders, 315,085.50, 49 with none.

C2 bi.category\_month: 144 rows, 316,023.50.

C3 bi.dim\_customer\_complete: 501 rows; INNER JOIN 6,951 lines, 316,023.50.

## HINT

C1 is the 3.3 query saved as a view, on top of bi.dim\_customer.

## TIMING

C1 in class. C2 in class if time, C3 after class.

CLOSE

# What you can do now

- Answer two-step questions with subqueries
- Write named steps with a CTE
- Save a query as a view

NEXT • D09

customer\_summary gives each customer's revenue. How would you rank the customers inside each segment without losing any rows?